RTOS-Based LED Blinking Control with Dynamic timing

Aim

Components

Connections - Circuit Diagram

Detailed Steps - Launching Code

Task - Exercise

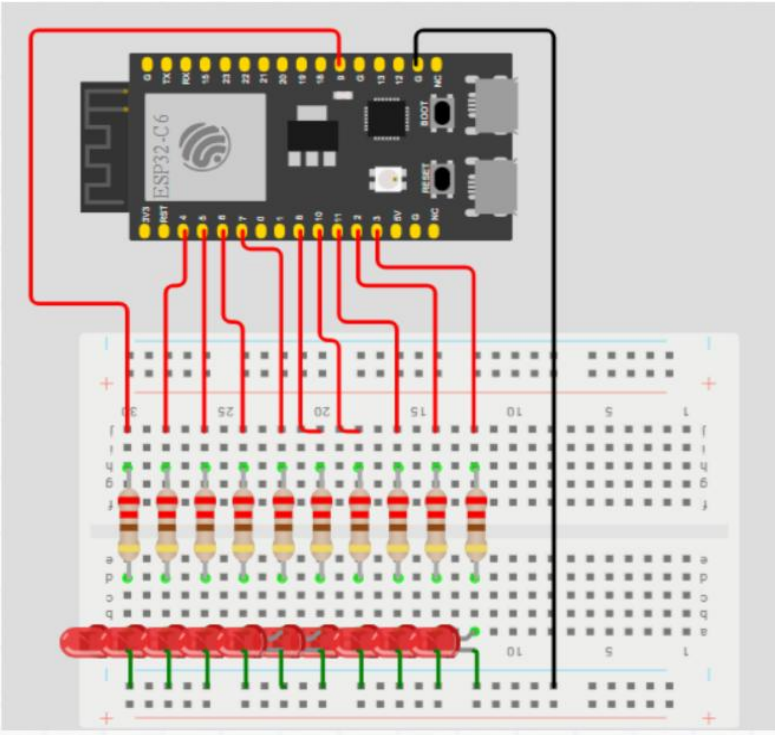
Aim:

To design and implement an RTOS-based LED control system, where two groups of LEDs blink with alternating delay patterns in a cyclic manner, demonstrating real-time task scheduling, precise timing control, and dynamic task execution using FreeRTOS."

Components:

Sno	Name of the Component	Quantity
1	ESP32 C6 Dev Module	1
2	Mini LED	10
3	220 Ohm Resistor	10
4	Breadboard	1
5	Jumper wires	As needed
6	USB to type C cable	1

Circuit Diagram



Detailed steps:

Step 1: Connect the longer leg of the LED to the resistor and the shorter leg to the ground terminal.

Step 2: Repeat the connections similarly for the remaining LEDs (10 in total) as shown in the circuit diagram. Ensure that the connections for each LED setup are in the same horizontal rows.

Step 3: The shorter terminals of the LEDs in the breadboard are connected to the Ground terminal of the ESP32-C6 development board using a jumper wire.

Step 4: The longer terminals of the LEDs with the resistors are connected to the GPIO pins of the ESP32C-6 using jumper wires. GPIO pin numbers are mentioned in the software code of the following pages.

Step 5: Check All Connections

Full code:

```
#include <Arduino.h>

#define GROUP_SIZE 5

int groupA[GROUP_SIZE] = {2, 3, 4, 5, 6};
int groupB[GROUP_SIZE] = {7, 8, 9, 10, 11};

bool reverseMode = false;  // Controls delay reversal

// Function to blink a group of LEDs
void blinkGroup(int *group, int delayTime, int duration) {
 unsigned long startTime = millis();

 while (millis() - startTime < duration) {
  for (int i = 0; i < GROUP_SIZE; i++) {
   digitalWrite(group[i], HIGH);
  }
  vTaskDelay(delayTime / portTICK_PERIOD_MS);

  for (int i = 0; i < GROUP_SIZE; i++) {
   digitalWrite(group[i], LOW);
  }
  vTaskDelay(delayTime / portTICK_PERIOD_MS);
 }
}
```

```
// RTOS Task
void ledTask(void *parameter) {
  while (1) {
    if (!reverseMode) {
      // Cycle 1
      blinkGroup(groupA, 10000, 22500); // 10s delay
      blinkGroup(groupB, 5000, 22500);  // 5s delay
    } else {
      // Cycle 2 (Reversed)
      blinkGroup(groupA, 5000, 22500);  // 5s delay
      blinkGroup(groupB, 10000, 22500); // 10s delay
    }

    reverseMode = !reverseMode; // Toggle mode after 45 seconds
  }
}

void setup() {
  for (int i = 0; i < GROUP_SIZE; i++) {
    pinMode(groupA[i], OUTPUT);
    pinMode(groupB[i], OUTPUT);
  }

  xTaskCreate(
    ledTask,     // Task function
    "LED Task",   // Task name
    4096,        // Stack size
    NULL,
    1,           // Priority
    NULL
  );
}

void loop() {
  // Empty – RTOS handles everything
}
```

Tasks:

1. RTOS Task Creation and Execution
Implement the FreeRTOS task for LED control on the ESP32-C6. Verify that the task executes continuously and independently of the main loop. Observe the blinking of Group A and Group B LEDs according to the specified delays (10 seconds and 5 seconds) and confirm that task scheduling and timing are functioning correctly.

2. Delay Timing Verification
Measure the actual ON and OFF durations of each LED group using a stopwatch or an oscilloscope. Compare the observed timing with the programmed delay values (10 seconds and 5 seconds). Confirm that the LED blinking pattern maintains accuracy over a complete 45-second cycle and discuss any discrepancies caused by task scheduling or execution latency.

3. Delay Reversal Functionality Test
After one complete cycle, verify that the delay pattern reverses as intended: Group A now blinks with 5 seconds delay and Group B with 10 seconds' delay. Observe multiple cycles to ensure that the reversal mechanism toggles correctly and consistently without manual intervention.

4. Multi-Cycle Stability Analysis
Run the LED sequence for an extended period (e.g., 5–10 cycles) and monitor the behavior of both LED groups. Confirm that the timing, reversal logic, and blinking pattern remain stable over time. Discuss any deviations caused by processor load, task preemption, or timing drift.

5. Interactive Control Implementation
Integrate push buttons to manually start, stop, or reverse the LED sequence. Verify that the system responds immediately to button inputs without disrupting the ongoing RTOS task execution. Analyze how the addition of user inputs affects task scheduling and real-time performance.

6. Task Prioritization and Preemption Analysis
Experiment by assigning different priorities to the LED tasks. Observe how FreeRTOS schedules tasks when multiple priorities are in effect. Document any changes in LED behavior, particularly when higher-priority tasks preempt lower-priority ones and discuss the impact of task priority on system performance.

7. LED Status Monitoring via Serial Output

Configure the system to send the status of each LED group (active/inactive and delay time) to the serial monitor. Verify that the output matches the actual LED blinking pattern. Discuss how serial communication interacts with task execution and whether it introduces any timing variation or delays**.**

RTOS-Based Multitasking System

Aim

Components

Connections - Circuit Diagram

Detailed Steps - Launching Code

Task - Exercise

Aim:

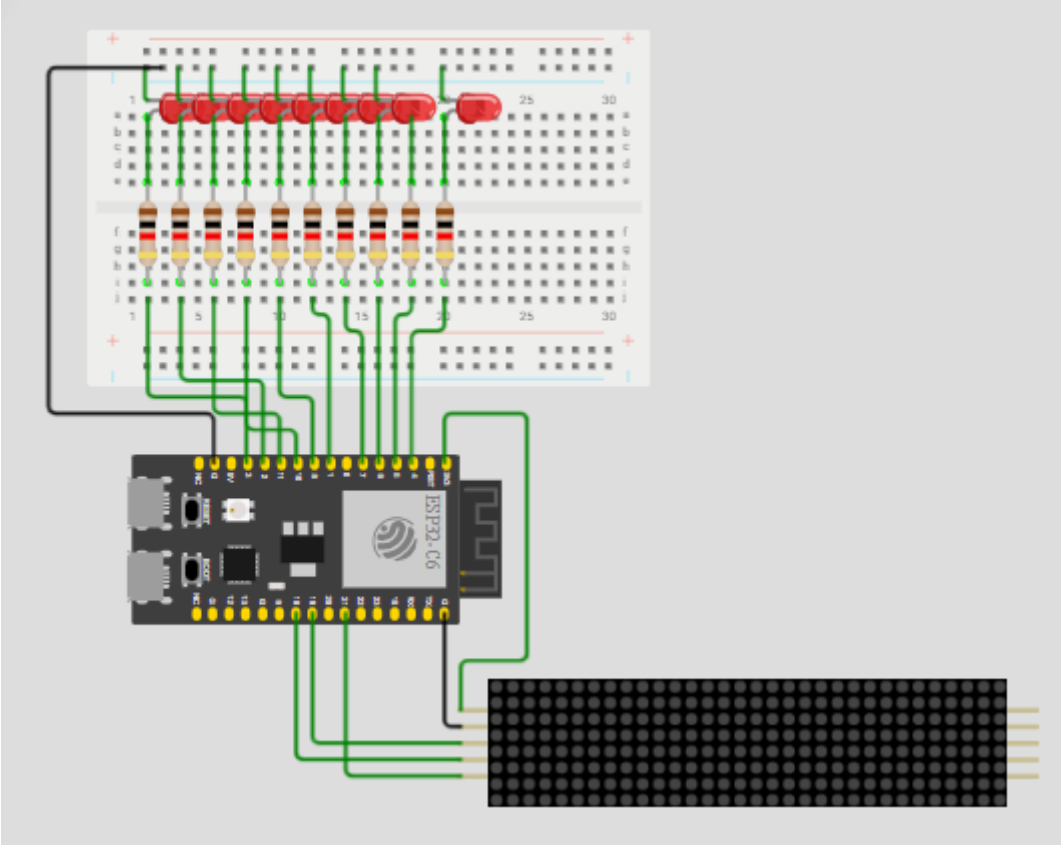
To design and implement an RTOS-based system using ESP32-C6 that demonstrates task scheduling by executing multiple tasks simultaneously, such as LED blinking and rolling display control.

Components Required:

Sno	Name of the Component	Quantity
1	ESP32 C6 Dev Module	1
2	Mini LED	10
3	220 Ohm Resistor	10
4	Breadboard	1
5	Jumper wires	As needed
6	USB to type C cable	1

Connections

Circuit Diagram



Detailed Steps:

Step 1: Place the ESP32-C6 development board on the breadboard and ensure it is properly powered via USB.

Step 2: Insert multiple LEDs (as shown) on the breadboard. Connect the longer leg (anode) of each LED to one end of a resistor.

Step 3: Connect the other end of each resistor to individual GPIO pins of the ESP32-C6 using jumper wires (as per your code pin configuration).

Step 4: Connect the shorter leg (cathode) of all LEDs to the ground (GND) rail of the breadboard.

Step 5: Connect the GND rail of the breadboard to the GND pin of the ESP32-C6.

Step 6: Place the LED matrix display module on the breadboard or externally as shown.

Step 7: Connect the VCC pin of the LED matrix to the 5V or 3.3V of the ESP32-C6.

Step 8: Connect the GND pin of the LED matrix to the GND of the ESP32-C6.

Step 9: Connect the control pins of the LED matrix (DIN, CS, CLK) to the respective GPIO pins of the ESP32-C6 (as defined in your program).

Step 10: Ensure all common grounds (LEDs, matrix, ESP32-C6) are properly connected.

Step 11: Verify all wiring connections carefully to avoid short circuits.

Step 12: Upload the code to the ESP32-C6 and test the LED blinking along with the rolling display functionality.

Full Code:

```
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Max72xxPanel.h>

// ================= SPI PINS (ESP32-C6) =================
#define DIN_PIN  19
#define CLK_PIN  21
#define CS_PIN   18

// ================= MATRIX SETUP =================
const int numberOfHorizontalDisplays = 4;
const int numberOfVerticalDisplays = 1;

Max72xxPanel matrix = Max72xxPanel(CS_PIN,
                      numberOfHorizontalDisplays,
```

```
                    numberOfVerticalDisplays);

// ================= LED SETUP =================
int ledPins[10] = {2, 4, 5, 8, 9, 10, 11, 12, 13, 14};

// ================= TASK DECLARATIONS =================
void Task_LED(void *pvParameters);
void Task_Display(void *pvParameters);

// ================= SETUP =================
void setup()
{
 Serial.begin(115200);

 // SPI initialization (MANDATORY for ESP32-C6)
 SPI.begin(CLK_PIN, -1, DIN_PIN, CS_PIN);

 // LED initialization
 for (int i = 0; i < 10; i++)
 {
  pinMode(ledPins[i], OUTPUT);
 }

 // Matrix initialization
 matrix.setIntensity(10);
 for (int i = 0; i < numberOfHorizontalDisplays; i++)
 {
  matrix.setPosition(i, i, 0);
  matrix.setRotation(i, 1);
 }

 // Create RTOS Tasks
 xTaskCreate(Task_LED, "LED Task", 1024, NULL, 1, NULL);
 xTaskCreate(Task_Display, "Display Task", 2048, NULL, 1, NULL);
}

void loop()
{
 // RTOS runs tasks, nothing here
}

// ================= LED TASK =================
void Task_LED(void *pvParameters)
{
 while (1)
 {
  for (int i = 0; i < 10; i++)
```

```
   {
    digitalWrite(ledPins[i], HIGH);
    vTaskDelay(100 / portTICK_PERIOD_MS);
    digitalWrite(ledPins[i], LOW);
   }
  }
}

// ================= DISPLAY TASK =================
void Task_Display(void *pvParameters)
{
  String text = "ESP32-C6 RTOS ";

  while (1)
  {
   int width = text.length() * 6;
   for (int x = matrix.width(); x > -width; x--)
   {
    matrix.fillScreen(LOW);
    matrix.setCursor(x, 0);
    matrix.print(text);
    matrix.write();
    vTaskDelay(80 / portTICK_PERIOD_MS);
   }
  }
}
```

Tasks:

1. RTOS Task Creation for LED and Display

Implement two separate FreeRTOS tasks on the ESP32-C6: one for LED blinking and another for controlling the rolling display. Verify that both tasks execute independently and simultaneously without blocking each other.

2. Concurrent Task Execution Verification

Observe the system to confirm that LED blinking and rolling display operate at the same time. Ensure that neither task interrupts or delays the execution of the other, demonstrating true multitasking.

3. Task Timing Accuracy Check

Measure the timing of LED blinking intervals and the speed of the rolling display. Compare the observed behavior with the programmed delays and verify the accuracy of RTOS scheduling.

4. Task Priority Experimentation

Assign different priorities to the LED and display tasks. Observe how the system

behaves when one task has higher priority than the other and analyze the effect on execution and responsiveness.

5. Task Synchronization

Implement synchronization mechanisms (like semaphores or queues) between tasks, if needed. Verify that data sharing or coordination between LED and display tasks works correctly without conflicts.